

AULA 1: FUNDAMENTOS DE DESENVOLVIMENTO

Lógica & Python

- Variáveis, Condicionais e tratamento de exceções.
- Pisando no acelerador com um Copiloto (IA generativa)

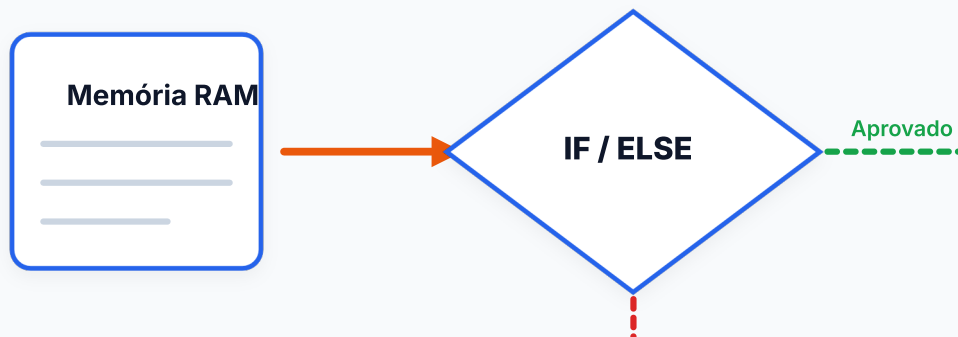
Menu da Aula



O Cenário de Negócio

A diretoria financeira identificou um gargalo crítico nas operações empresariais: os pedidos de reembolso de despesas de viagens corporativas chegam por e-mail sem padrão estruturado. Isso exige horas de auditoria manual e cálculos exaustivos em planilhas pela equipe de RH.

Nossa Missão: Construir um sistema corporativo de auditoria de despesas focado em *Compliance*. A aplicação deverá receber os dados inseridos pelo funcionário, processar as regras de limites de gastos por categoria da empresa e emitir um parecer imediato, otimizando o fluxo de trabalho administrativo.



Fundamentos Arquiteturais

Para estruturar este software do absoluto zero, recorreremos aos conceitos básicos da documentação oficial do Python. Dominaremos três pilares essenciais:

1. Variáveis e Tipagem Dinâmica

Analogia Prática: Imagine a memória do computador como um imenso arquivo de aço. Uma variável é como uma *gaveta etiquetada* onde você guarda um documento temporário. O identificador da gaveta é a variável, e o documento dentro é o valor.

O Python possui **Tipagem Forte**. O sistema não permite misturar tipos incompatíveis de dados (como somar palavras com números) sem antes fazer uma conversão (Casting) explícita pelo desenvolvedor.

2. Estruturas Lógicas de Decisão (IF / ELIF / ELSE)

Analogia Prática: Pense nesta estrutura como um fluxograma de triagem. O código testa uma condição principal (**IF** o gasto for transporte). Se for verdade, ele executa a regra e encerra. Se não for, ele avalia uma condição alternativa (**ELIF** alimentação), ou aplica a política residual (**ELSE** despesa negada).

3. Resiliência de Software (Try / Except / Finally)

Sistemas corporativos quebram facilmente devido a entradas imprevisíveis dos usuários. Se o sistema pede o valor numérico de um hotel e o usuário digita a palavra "oitocentos", o interpretador entra em colapso. Envolver a lógica matemática em um bloco try/except é como instalar um "airbag": se ocorrer o choque de dados inválidos, o erro é



interceptado e o sistema informa o usuário elegantemente sem fechar abruptamente.